Recap

- Why is **DeepSeek V3** good and cheap?
  - Mixture of Experts Architecture
  - Multihead Latent Attention
  - Multi-Token Prediction
  - Mixed Precision Training
  - Blockwise Quantization
  - Tensor and CUDA core optimizations
  - Etc

- Modern LLM Training
  - Pre-Training
  - Post-Training
    - SFT
    - RLHF
      - PPO
      - DPO
      - GRPO

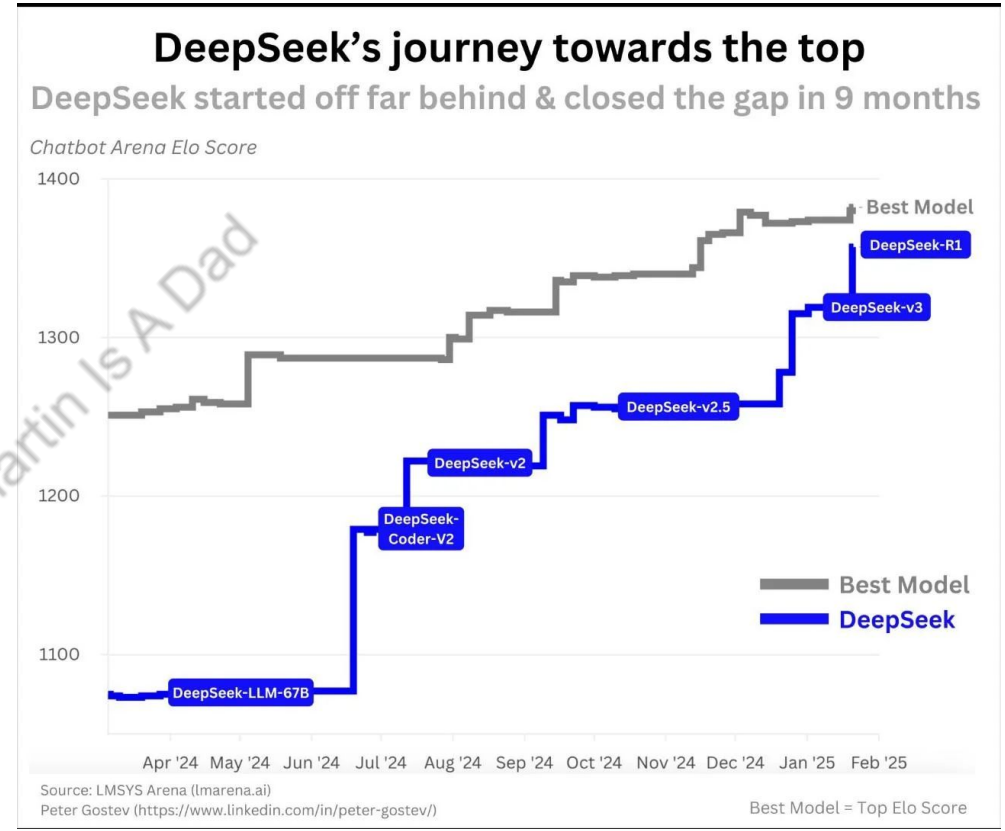
- RL Basics

DeepSeek's Evolution

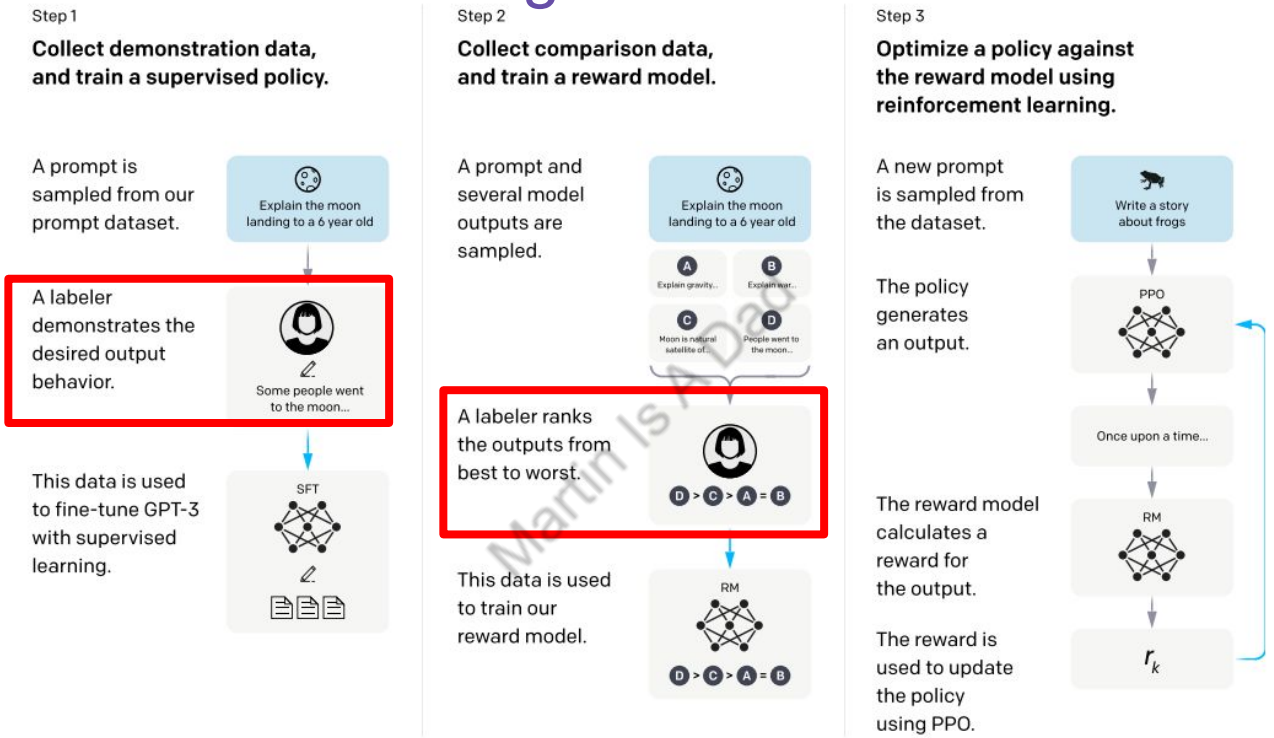
- We have covered DeepSeek V3
- What about the best one yet, **DeepSeek R1?**

DeepSeek Papers

- DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning
- DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models
- DeepSeek-V3 Technical Report



Modern LLM Post-Training



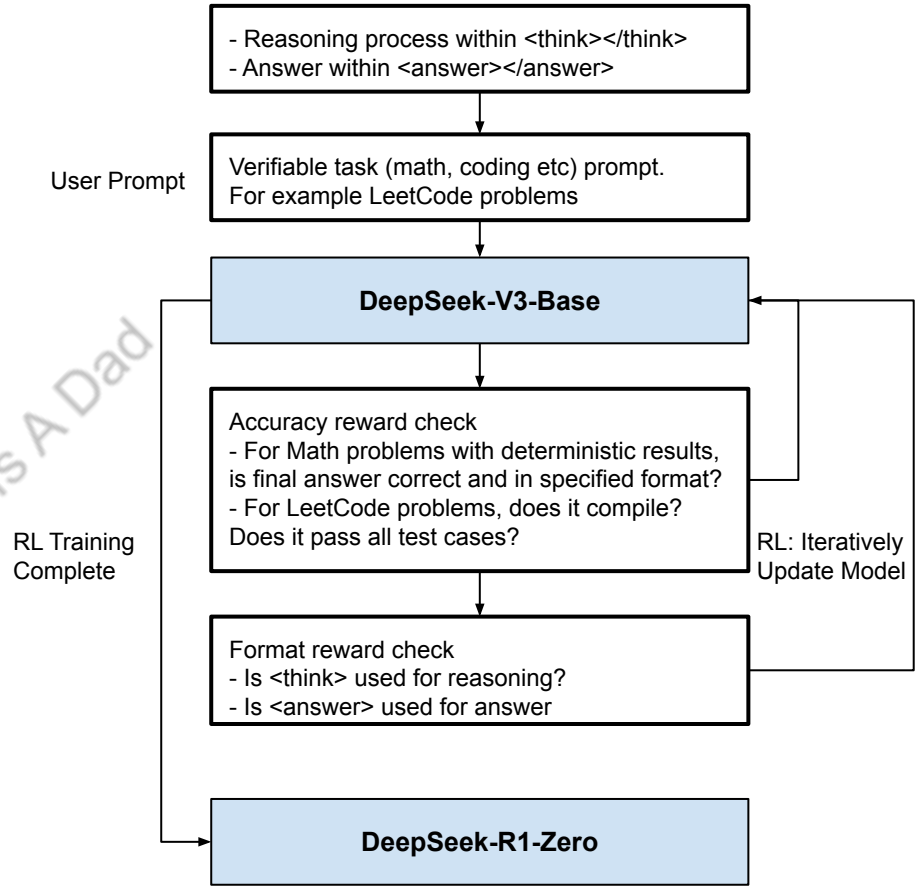
- Places where we need human interaction: (1) SFT (2) Reward Model Training
- Can we get rid of those, since dependency on human is
  - Bottleneck for scaling
  - Could have bias

DeepSeek R1-Zero

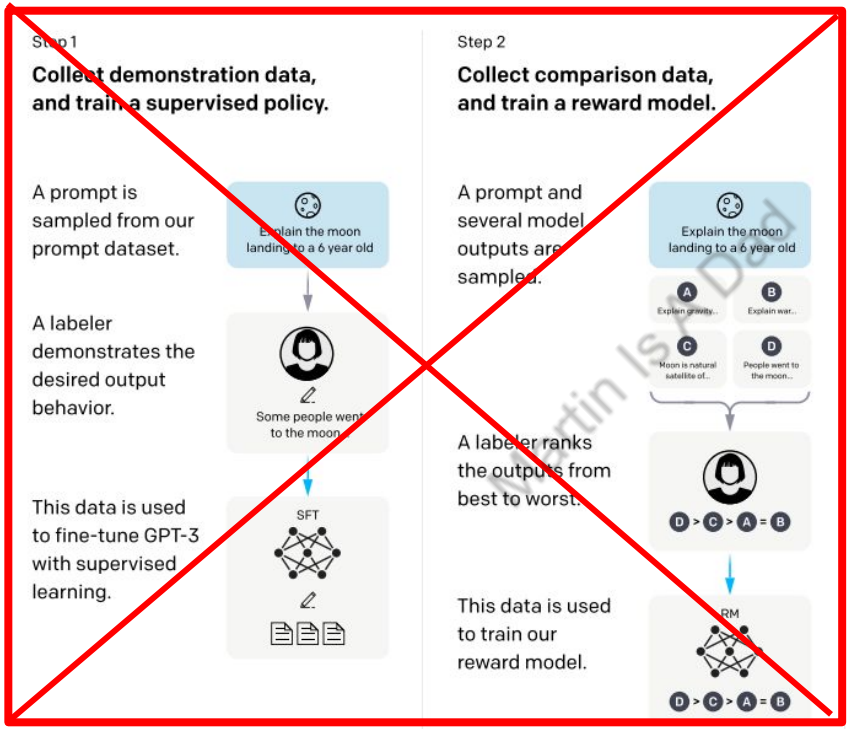
The answer is **YES**!

Specs

- Pre-Training base model is DeepSeek-V3-Base
- No Supervised Fine Tuning
- No reward model training, instead do rule based rewards:
  - Accuracy rewards: whether answer is correct
  - Format rewards: give additional reward to <think>
- Question 1: How to know whether LLM is giving right answer?
  - Answer: focus on tasks that's verifiable (math, coding etc)
- Question 2: Why not a neural model for reward model?
  - Answer: Neural reward model suffer from reward hacking in large scale RL training, and it needs additional training resources (computation, memory), complex whole training pipeline.



DeepSeek R1-Zero



Can we do better here too?

GRPO vs PPO Simplified

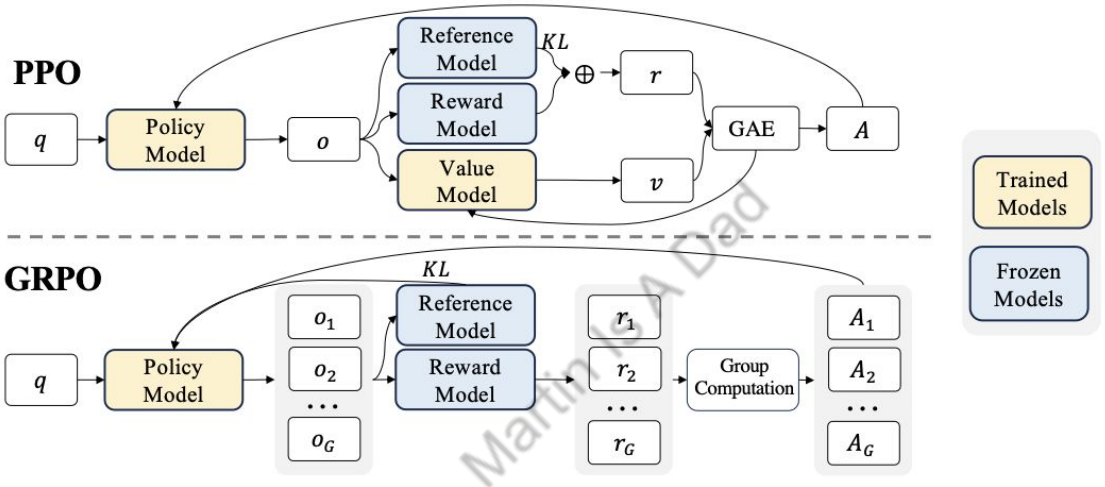


Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

- PPO requires a trained value model (V(s)) to estimate the “average reward” given a state, later used to compute advantage: Advantage = Reward - V(s)
- GRPO used group sampling to estimate the “average” reward. In practice the sampling pool size is ~16

GRPO vs PPO Deep Dive

- Both PPO and GRPO are Policy Optimization, meaning we are optimizing the policy directly. Policy is the operation strategy of the LLM, or the LLM itself.
- Goal: maximize expected total reward $J(\pi_\theta) = \mathop{\mathrm{E}}_{\tau \sim \pi_\theta}[R(\tau)]$
- PPO (can work with or without KL regularization, this one is without)

$$J(\theta) = \frac{1}{G}\sum_{i=1}^{G} \min\left(\frac{\pi_\theta(a_i|s)}{\pi_{\theta_{old}}(a_i|s)}A_i, \text{clip}\left(\frac{\pi_\theta(a_i|s)}{\pi_{\theta_{old}}(a_i|s)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right))$$

- GRPO

$$J(\theta) = \frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(a_i|s)}{\pi_{\theta_{old}}(a_i|s)}A_i, \text{clip}\left(\frac{\pi_\theta(a_i|s)}{\pi_{\theta_{old}}(a_i|s)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right) - \beta D_{KL}(\pi_\theta||\pi_{ref})\right)$$

GRPO vs PPO Deep Dive

- PPO $$J(\theta) = \frac{1}{G}\sum_{i=1}^{G} \min\left(\frac{\pi_\theta(a_i|s)}{\pi_{\theta_{old}}(a_i|s)}A_i, \text{clip}\left(\frac{\pi_\theta(a_i|s)}{\pi_{\theta_{old}}(a_i|s)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right))$$

- Policy updates via $$\theta_{k+1} = \arg\max_\theta \mathop{\mathrm{E}}_{s,a\sim\pi_{\theta_k}} \left[L(s,a,\theta_k,\theta)\right],$$

$$L(s,a,\theta_k,\theta) = \min\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}A^{\pi_{\theta_k}}(s,a),\ \text{clip}\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}, 1-\epsilon, 1+\epsilon\right)A^{\pi_{\theta_k}}(s,a)\right)$$

- TLDR: for each RL training iteration, we want the set of parameters that can maximize *J*, capped by a ceiling ε (new model can't be too different than previous). Advantage here is calculated with the trained value/critic model: Advantage = Reward - V(s)

GRPO vs PPO Deep Dive

- GRPO is very similar with PPO, with the following differences:
  - Eliminates the need for a value network for less memory/compute
  - Uses group sampling for more stable advantage estimation

$$J(\theta) = \frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(a_i|s)}{\pi_{\theta_{old}}(a_i|s)}A_i, \text{clip}\left(\frac{\pi_\theta(a_i|s)}{\pi_{\theta_{old}}(a_i|s)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right) - \beta D_{KL}(\pi_\theta||\pi_{ref})\right)$$

$$A_i = \frac{r_i - \text{mean}(r_1, r_2, \cdots, r_G)}{\text{std}(r_1, r_2, \cdots, r_G)}$$

$$\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1,$$

- Advantage calculation is using Z-Score of a group, similar to Batch/Layer Norm in Transformer!
- Again similar to PPO, Policy updates via $\theta_{k+1} = \arg\max_\theta \underset{s,a\sim\pi_{\theta_k}}{\mathrm{E}}\left[L(s,a,\theta_k,\theta)\right],$
  - Still capped by ceiling ε

DeepSeek R1-Zero

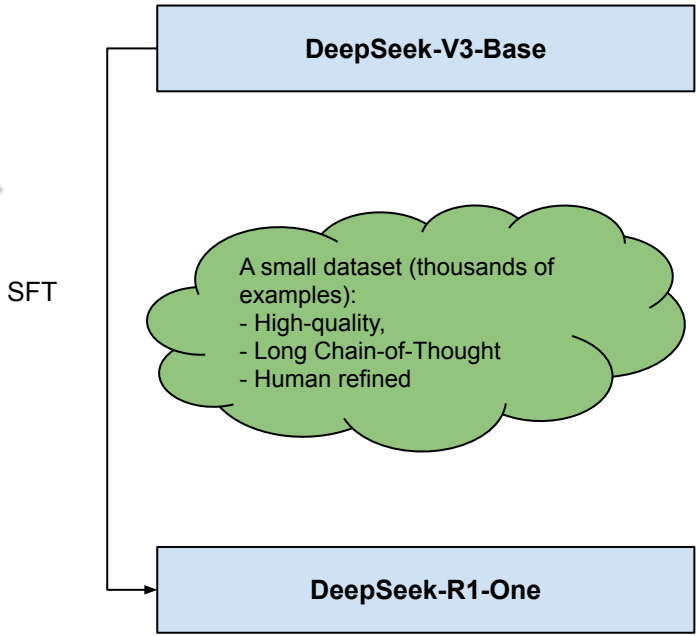
- Results show that pure RL trained R1-Zero shows structure reasoning abilities
- SFT and RLHF have been the dominant training methods for LLM. The underlying assumption is human-labeled data is necessary for structured reasoning, RL without human preference modeling can't achieve the same
- DeepSeek R1-Zero proves this assumption **wrong**!
- Drawbacks of R1-Zero:
  - Bad readability: long and unstructured responses
  - Language mixing: responses contained multiple languages in a single answer
  - Bad formatting: no markdown, no clear separation between thoughts and conclusions
- DeepSeek R1 created with motivation to perfect R1-Zero

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a+x}} = x$ is equal to
Response: <think> To solve the equation $\sqrt{a - \sqrt{a+x}} = x$, let's start by squaring both $\cdots$ $\left(\sqrt{a - \sqrt{a+x}}\right)^2 = x^2 \implies a - \sqrt{a+x} = x^2.$ Rearrange to isolate the inner square root term: $(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$ $\ldots$ Wait, wait. Wait. That's an aha moment I can flag here. Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$ We started with the equation: $\sqrt{a - \sqrt{a+x}} = x$ First, let's square both sides: $a - \sqrt{a+x} = x^2 \implies \sqrt{a+x} = a - x^2$ Next, I could square both sides again, treating the equation: $\cdots$ $\ldots$

Table 3 | An interesting "aha moment" of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

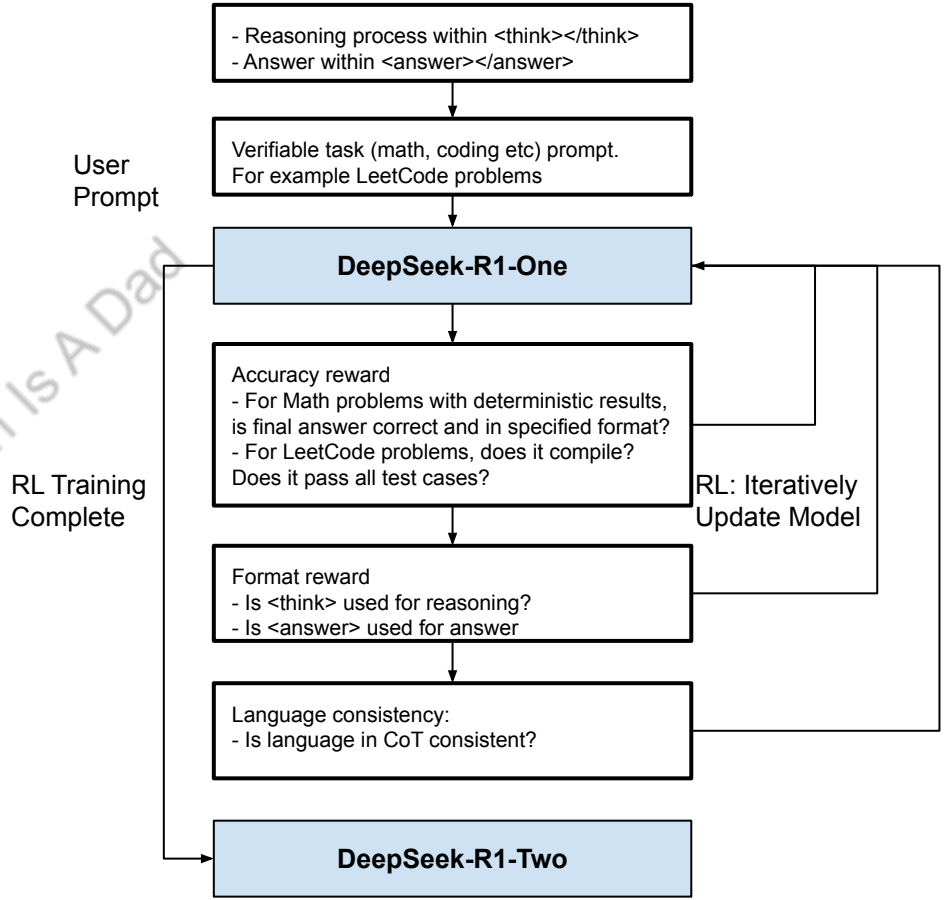
DeepSeek R1 Stage #1: Cold Start

- **Goal:** Push model towards better reasoning and more readable output, avoiding the instability of pure RL from scratch.
- **Method:** Minimal supervised fine tuning on DeepSeek-V3-Base
- A small dataset (thousands of examples) of high-quality, long Chain-of-Thought (CoT) data is curated. This data is generated through a mix of few-shot prompting, asking models to generate detailed answers with reflection, and refining outputs (by human annotators) from DeepSeek-R1-Zero.
- Advantages (compared with R1-Zero):
  - Better Readability
  - Better Performance



DeepSeek R1 Stage #2: RL with GRPO

- **Goal:**
  - Enhance model's reasoning capabilities
  - Improve language consistency
- **Method:** RL with GRPO on DeepSeek-R1-One
- Similar with training DeepSeek-R1-Zero. Still ran into language mixing especially in CoT, introduced language consistency reward in RL.



DeepSeek R1 Stage #3: Rejection Sampling

- Rejection sampling is a method used to generate samples from a complex probability distribution by drawing samples from a simpler, "proposal" distribution and then accepting or rejecting them based on a probability related to the ratio of the target and proposal densities.

- Steps (assume target distribution is f(x))
  1. Choose a proposal distribution (g(x))
  2. Sample from the proposal distribution (g(x))
  3. Calculate a acceptance probability
  4. Accept or reject the sample
  5. Repeat until samples are enough

- Steps in LLM context:
  1. Generate Candidate Outputs: Use the LLM to generate a batch of K candidate responses for a given prompt.
  2. Evaluate with a Reward Model: Feed each candidate response to reward model. The reward model outputs a scalar value representing the quality of the response.
  3. Select the Best: Select the candidate with the highest reward score.
  4. Fine-tune the LLM: Use the selected candidate to fine-tune the LLM.
  5. Repeat

DeepSeek R1 Stage #3: Rejection Sampling

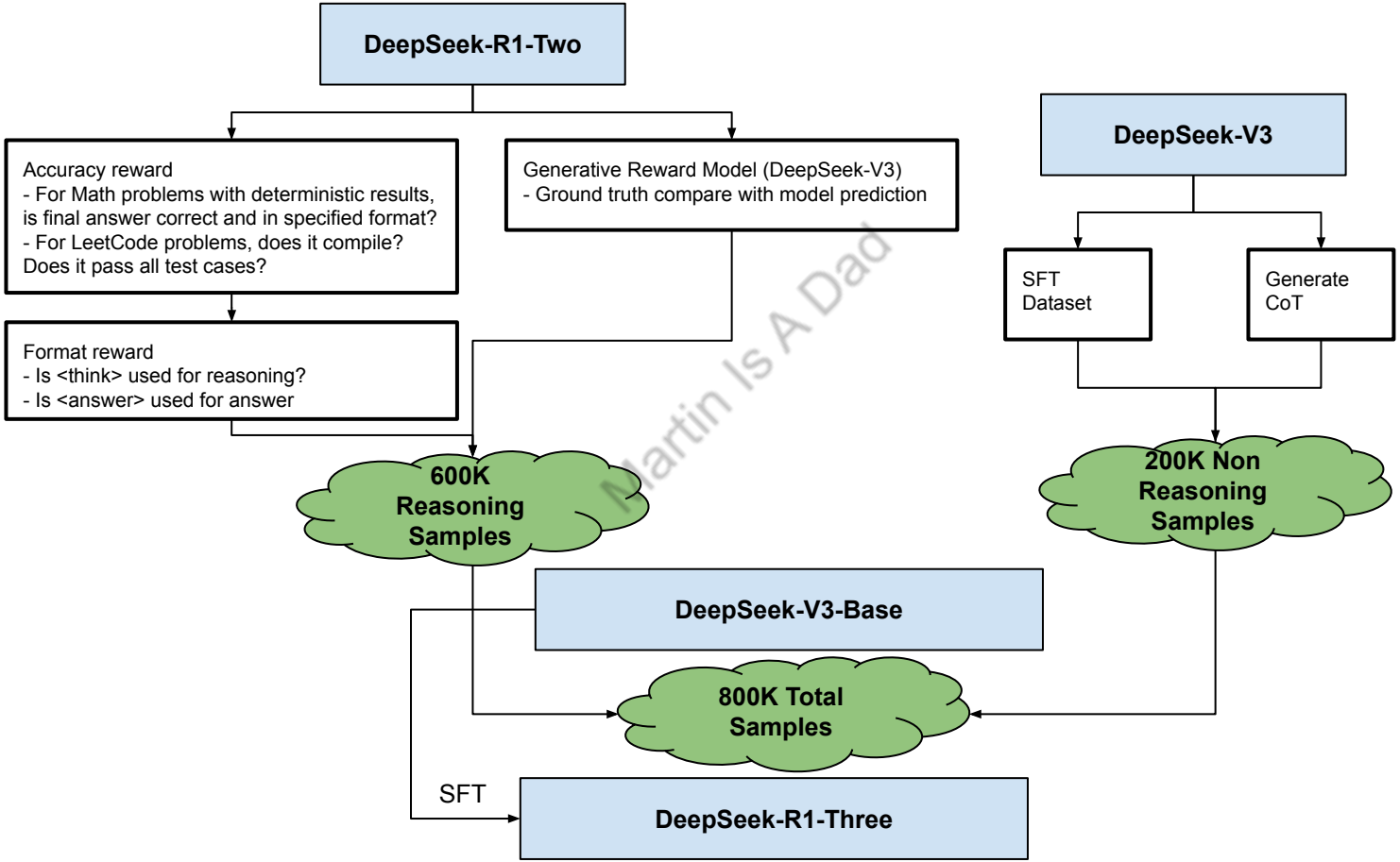
- **Goal:** Create a high-quality SFT dataset beyond ***verifiable*** reasoning, including writing, role-playing, and other general-purpose tasks

- **Method:** Rejection sampling on DeepSeek-R1-Two

- Reasoning data:
  - Expand beyond data that can be verified by rules
  - Expanded data are rated by generative reward model, by feeding ground truth and predictions into DeepSeek-V3
  - Filter out CoTs with bad readability

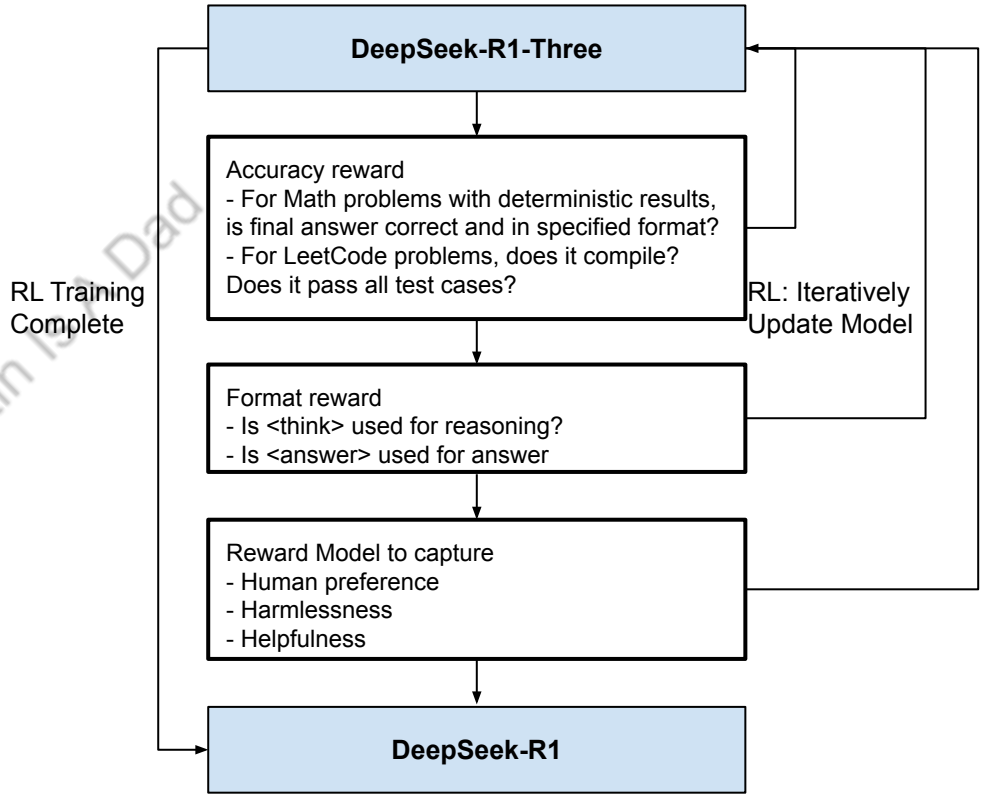
- Non Reasoning data:
  - SFT data for DeepSeek-V3 is reused
  - DeepSeek-V3 used to generated non reasoning examples

DeepSeek R1 Stage #3: Rejection Sampling



DeepSeek R1 Stage #4: RL for All Scenarios

- **Goal:** Align model with human preferences (helpfulness and harmlessness) and further refining reasoning.
- **Method:** RL with GRPO on DeepSeek-R1-Three
- **Reward model:**
  - Rule-based rewards for reasoning tasks, similar with R1-Zero and stage#2
  - **Learned reward models** to capture human preferences in complex scenarios. For helpfulness and harmlessness, focusing on the final summary for helpfulness and on the entire response for harmlessness



DeepSeek R1 Distillation

- **Goal:** Allow more efficient smaller models with same/similar level of reasoning capabilities like DeepSeek-R1

- **How Distillation Works**
  - (From previous steps) Reuse 800K high-quality SFT samples
  - These samples are then used to fine-tune smaller models like Llama to replicate the larger model's reasoning.
  - No RL is applied to the distilled models, even though incorporating RL could further enhance performance. **This enables smaller models to retain advanced reasoning skills without the high cost of training from scratch.**

- Result: This straightforward distillation method significantly enhances the reasoning abilities of smaller models